

Lesson: Stack and Queue Abstract Data Types with Linked List Operations in C++ (Line-by-Line Explanation)

In this lesson, we'll cover the **Stack** and **Queue** abstract data types (ADTs) and their implementations using linked lists. I'll provide line-by-line explanations to help you understand how each part of the code works.

1. Stack ADT with Linked List

Stack Operations:

- **Push**: Add an element to the top of the stack.
- **Pop**: Remove the element from the top of the stack.
- **Peek**: Return the top element without removing it.
- **IsEmpty**: Check if the stack is empty.

Implementing Stack using Linked List

```
#include <iostream>    // Include necessary library for input and output
using namespace std;   // Use the standard namespace for convenience

// Node structure for the linked list
struct Node {
    int data;           // Data held by the node
    Node* next;         // Pointer to the next node in the stack
};

// Stack class
class Stack {
private:
    Node* top;          // Pointer to the top of the stack

public:
    Stack() : top(nullptr) {} // Constructor initializes the stack with top as nullptr

    // Push operation to add an element to the top of the stack
    void push(int value) {
        Node* newNode = new Node(); // Create a new node
        newNode->data = value;        // Set the data of the new node
        newNode->next = top;          // Point the new node to the current top
        top = newNode;               // Update top to the new node
        cout << value << " pushed to stack\n";
    }
}
```

```

// Pop operation to remove the element from the top of the stack
int pop() {
    if (isEmpty()) {                // Check if the stack is empty
        cout << "Stack is empty!\n";
        return -1;                  // Return -1 if the stack is empty
    }
    int value = top->data;           // Store the data of the top node
    Node* temp = top;               // Store the top node in a temporary variable
    top = top->next;                 // Update top to the next node
    delete temp;                   // Delete the old top node
    cout << value << " popped from stack\n";
    return value;                   // Return the value that was popped
}

// Peek operation to return the top element without removing it
int peek() {
    if (isEmpty()) {                // Check if the stack is empty
        cout << "Stack is empty!\n";
        return -1;                  // Return -1 if the stack is empty
    }
    return top->data;               // Return the data of the top node
}

// Check if the stack is empty
bool isEmpty() {
    return top == nullptr;          // Return true if top is nullptr
}
};

// Main function to test the stack
int main() {
    Stack stack;                    // Create a new stack instance
    stack.push(10);                 // Push 10 onto the stack
    stack.push(20);                 // Push 20 onto the stack
    cout << "Top element is: " << stack.peek() << "\n"; // Peek at the top element
    stack.pop();                    // Pop the top element from the stack
    cout << "Top element is: " << stack.peek() << "\n"; // Peek again at the new top element
    return 0;
}

```

Explanation of Key Parts:

1. **Node Structure:** Each node contains data and a pointer to the next node.
2. **Stack Class:** Contains a private pointer `top` and methods for stack operations.
3. **Push Method:** Adds a new node to the top, linking it to the current top node.
4. **Pop Method:** Removes the top node and updates the `top` pointer.

5. **Peek and isEmpty:** Allow you to view the top element and check if the stack is empty.
-

2. Queue ADT with Linked List

Queue Operations:

- **Enqueue:** Add an element to the end of the queue.
- **Dequeue:** Remove the element from the front of the queue.
- **Peek:** Return the front element without removing it.
- **IsEmpty:** Check if the queue is empty.

Implementing Queue using Linked List

```
#include <iostream>    // Include necessary library for input and output
using namespace std;   // Use the standard namespace for convenience

// Node structure for the linked list
struct Node {
    int data;           // Data held by the node
    Node* next;         // Pointer to the next node in the queue
};

// Queue class
class Queue {
private:
    Node* front;        // Pointer to the front of the queue
    Node* rear;         // Pointer to the rear of the queue

public:
    Queue() : front(nullptr), rear(nullptr) {} // Constructor initializes front and rear to nullptr

    // Enqueue operation to add an element to the end of the queue
    void enqueue(int value) {
        Node* newNode = new Node(); // Create a new node
        newNode->data = value;       // Set the data of the new node
        newNode->next = nullptr;     // The new node will be the last, so its next is nullptr

        if (isEmpty()) {             // If the queue is empty, both front and rear are the new
node
            front = rear = newNode;
        } else {
            rear->next = newNode;    // Link the current rear to the new node
            rear = newNode;          // Update rear to the new node
        }
    }
}
```

```

        cout << value << " added to queue\n";
    }

    // Dequeue operation to remove the element from the front of the queue
    int dequeue() {
        if (isEmpty()) {                // Check if the queue is empty
            cout << "Queue is empty!\n";
            return -1;                  // Return -1 if the queue is empty
        }
        int value = front->data;         // Store the data of the front node
        Node* temp = front;              // Store the front node in a temporary variable
        front = front->next;              // Update front to the next node

        if (front == nullptr) {          // If front becomes nullptr, set rear to nullptr too
            rear = nullptr;
        }
        delete temp;                    // Delete the old front node
        cout << value << " removed from queue\n";
        return value;                    // Return the value that was dequeued
    }

    // Peek operation to return the front element without removing it
    int peek() {
        if (isEmpty()) {                // Check if the queue is empty
            cout << "Queue is empty!\n";
            return -1;                  // Return -1 if the queue is empty
        }
        return front->data;              // Return the data of the front node
    }

    // Check if the queue is empty
    bool isEmpty() {
        return front == nullptr;        // Return true if front is nullptr
    }
};

// Main function to test the queue
int main() {
    Queue queue;                       // Create a new queue instance
    queue.enqueue(10);                  // Enqueue 10 to the queue
    queue.enqueue(20);                  // Enqueue 20 to the queue
    cout << "Front element is: " << queue.peek() << "\n"; // Peek at the front element
    queue.dequeue();                    // Dequeue the front element from the queue
    cout << "Front element is: " << queue.peek() << "\n"; // Peek again at the new front element
    return 0;
}

```

Explanation of Key Parts:

1. **Node Structure:** Each node contains data and a pointer to the next node.
2. **Queue Class:** Contains two pointers (**front** and **rear**) to facilitate efficient insertion and deletion.
3. **Enqueue Method:** Adds a new node to the rear of the queue.
4. **Dequeue Method:** Removes the node at the front, and updates **front** and **rear** as necessary.
5. **Peek and isEmpty:** Allow you to view the front element and check if the queue is empty.

In both cases, the use of a linked list allows the stack and queue to grow dynamically, without any predefined size limit. This makes these data structures efficient and flexible for a variety of applications.